

1 Plug-In Interface

Below follows the definition of the Plug-In Interface. It consists of a number of C++ routines for the transfer of data between TRIBON Nesting and the external sequence generator.

WO101

NAME:

WO101 - AutoSeq Interface routine - Is External
AutoSeq Connected

PURPOSE:

The procedure checks if an external AutoSeq
algorithm has been connected.

DECLARATION:

```
int wo101 ( void);
```

RESULT:

The procedure returns TRUE if connected, otherwise
FALSE.

-----*/

```

/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
*
*   Main procedure - wo101
*
*****
*/

#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        int wo101 ( void)
        {
            return 1;
        }
}

```

WO102

NAME:

WO102 - AutoSeq Interface routine - Add Nesting Info

PURPOSE:

The procedure adds information for one nesting.

DECLARATION:

```
void wo102( const char *NestName,
            const int NestType,
            const double Length,
            const double Width,
            const double Thickness,
            const char *Quality,
            const int Nparts,
            const int ContourDirection,
            const int HoleDirection,
            const int StartCode,
            const int StartEndAct,
            const double StartNode,
            const double BridgeWidth,
            const double BridgeWidthHole,
            const double StartPosition[2],
            const double EndPosition[2],
            const double EndMarkPosition[2],
            const int MarkingFirst,
            const int MarkingOpposite,
            const double HoleStart,
            const int HoleStartType);
```

INPUT:

Parameters:

NestName	The name of the nesting
NestType	The type of nesting: = 0 Normal = 1 NC = 2 FP
Length	The raw plate length
Width	The raw plate width
Thickness	The raw plate thickness
Quality	The raw plate quality

Nparts	The number of nested parts
ContourDirection	The cutting direction of the outer contours (Nesting default CONT_DIRECTION)
HoleDirection	The cutting direction of the holes (Nesting default HOLE_DIRECTION)
StartCode	The start code (Nesting default START_CODE)
StartEndAct	Start/end position code (Nesting default START_END_ACT)
StartNode	Start node preference (Nesting default START_NODE)
BridgeWidth	The bridge width (Nesting default BRIDGE_WIDTH)
BridgeWidthHole	The bridge width for holes (Nesting default BRIDGE_WIDTH_HOLE)
StartPosition	Start position (Nesting default START_POS)
EndPosition	End position (Nesting default END_POS)
EndMarkPosition	End marking position (Nesting default END_MARK_POS)
MarkingFirst	Partwise/platewise marking (Nesting default MARKING_ALL_FIRST)
MarkingOpposite	Automatic marking of other side (Nesting default MARK_AUTO_OPPOSITE)
HoleStart	Angle used to determine the starting point on a hole (Nesting default START_DIR_HOLE)
HoleStartType	The type of hole for which HoleStart shall be used (Nesting default START_DIR_HOLE_TYPE)

RESULT:

None.

```

----- */
/*
 *      External declarations
 */
extern "C"
{
#if !defined TBLSEQUENCEPLUGIN_API
#define TBLSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - wo102
 *
*****
 */
#ifdef WIN32
TBLSEQUENCEPLUGIN_API
#endif
        void wo102( const char * /* NestName */,
                    const int /* NestType */,
                    const double /* Length */,
                    const double /* Width */,
                    const double /* Thickness */,
                    const char * /* Quality */,
                    const int /* Nparts */,
                    const int /* ContourDirection */,
                    const int /* HoleDirection */,
                    const int /* StartCode */,
                    const int /* StartEndAct */,
                    const double /* StartNode */,
                    const double /* BridgeWidth */,
                    const double /* BridgeWidthHole */,
                    const double [2] /* StartPosition */,
                    const double [2] /* EndPosition */,
                    const double [2] /* EndMarkPosition */,
                    const int /* MarkingFirst */,
                    const int /* MarkingOpposite */,
                    const double /* HoleStart */,
                    const int /* HoleStartType */)
        {
        }
}

```

WO103

NAME:

WO103 - AutoSeq Interface routine - Add Contour Data

PURPOSE:

The procedure adds information for one contour.

DECLARATION:

```
void wo103( const int PlatePartNo,
            const int CntNo,
            const int ContourNo,
            const int NumberOfSegParts,
            const char *Assid1,
            const char *Assid2,
            const char *Assid3,
            const char *Assid4,
            const int ContourType,
            const int MarkingType,
            const int MarkingSide);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
CntNo	The contour number within the plate part, starting at 1
ContourNo	The contour number in the Nesting system
NumberOfSegParts	The number of segment parts for the contour
Assid1- Assid4	The assembly id's 1-4 from the General Purpose Strings (GPS) Irrelevant for burning contours.
ContourType	The type of contour: = 1 Outer contour = 2 Hole = 3 Marking = 4 Blasting = 5 GSD
MarkingType	The type of marking/blasting/GSD:

```

= 0    Hull curve or no reference
= 1    Hole
= 2    Bracket
= 3    Inner seam in PLCM
= 4    Stiffener
= 5    Flange
= 6    Added contour
= 7    Bracket notch
= 8    Knuckle line
= 9    Reference plane
= 10   Pipe penetration
= 11   Nominal contour
= 12   Web with flange geometry
= 13   Flange with web geometry
= 14   Structure
= 15   Swedging
= 16   Metal touching line
= 17   Small hole
= 18   Bracket overlap
= 19   Intersecting panel
= 20   Intersecting bracket on

another panel

= 21   Cutout crossing
= 22   Abutting profile
= 23   Abutting shell profile
= 24   Reference line in PLCM
= 25   Attachment angle,

intersecting panel

= 26   Attachment angle,

intersecting bracket

= 27   Attachment angle, profile
= 28   Attachment angle, bracket
= 29   GSD at plate part corner
= 30   GSD, manually added
= 31   GSD at notches for
       seams/butts in the shell
= 32   GSD at CL

MarkingSide    The side of
                marking/blasting/GSD:
= +1    This side
= 0     Both sides
= -1    Other side
Irrelevant for burning contours.

```

RESULT:

None.

```

-----*/
/*
*      External declarations
*/
extern "C"
{

```

```

#if !defined TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
*
*   Main procedure - wo103
*
*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif

void wo103( const int /* PlatePartNo */,
            const int /* CntNo */,
            const int /* ContourNo */,
            const int /* NumberOfSegParts */,
            const char * /* Assid1 */,
            const char * /* Assid2 */,
            const char * /* Assid3 */,
            const char * /* Assid4 */,
            const int /* ContourType */,
            const int /* MarkingType */,
            const int /* MarkingSide */)
{
    {
    }
}

```


WO104

NAME:

WO104 - AutoSeq Interface routine - Define Number of
Bevel Intervals

PURPOSE:

The procedure defines the number of bevel intervals
for the current contour.

DECLARATION:

```
void wo104( const int PlatePartNo,  
            const int ContourNo,  
            const int NumberOfBevels);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
NumberOfBevels	The number of bevel intervals (only relevant for outer contours and holes).

RESULT:

None.

```
-----*/  
/*  
 *      External declarations  
 */  
extern "C"  
{  
#if !defined TBLSEQUENCEPLUGIN_API  
#define TBLSEQUENCEPLUGIN_API  
#endif  
  
/*  
*****  
 *  
 *  Main procedure - wo104  
 *  
 */
```

```
*****
*/

#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif

void w0104( const int /* PlatePartNo */,
            const int /* ContourNo */,
            const int /* NumberOfBevels */)
{
}
}
```

WO105

NAME:

WO105 - AutoSeq Interface routine - Add Bevel
Interval for Contour

PURPOSE:

The procedure adds a bevel interval for the current
contour.

DECLARATION:

```
void wo105( const int PlatePartNo,  
            const int ContourNo,  
            const int BevelIntervalNo,  
            const int SegPartStart,  
            const double StartPoint[2],  
            const int SegPartEnd,  
            const double EndPoint[2],  
            const double BevelCode);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
BevelIntervalNo	The bevel interval number
SegPartStart	Segment part number for the bevel interval start
StartPoint	Interval starting point
SegPartEnd	Segment part number for the bevel interval end
EndPoint	Interval ending point
BevelCode	The bevel code (< 0 indicates other side)

RESULT:

None.

```

-----*/
/*
 *      External declarations
 */
extern "C"
{
#if !defined TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - wo105
 *
*****
 */
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        void wo105( const int /* PlatePartNo */,
                    const int /* ContourNo */,
                    const int /* BevelIntervalNo */,
                    const int /* SegPartStart */,
                    const double [2] /* StartPoint */,
                    const int /* SegPartEnd */,
                    const double [2] /* EndPoint */,
                    const double /* BevelCode */)
        {
        }
}

```

WO106

NAME:

WO106 - AutoSeq Interface routine - Define Number of
Cutouts/Notches

PURPOSE:

The procedure defines the number of cutouts/notches
for the current contour.

DECLARATION:

```
void wo106( const int PlatePartNo,  
            const int ContourNo,  
            const int NumberOfCutouts);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
NumberOfCutouts	The number of cutouts/notches (only relevant for outer contours).

RESULT:

None.

```
-----*/  
/*  
 *      External declarations  
 */  
extern "C"  
{  
#if !defined TBLSEQUENCEPLUGIN_API  
#define TBLSEQUENCEPLUGIN_API  
#endif  
  
/*  
*****  
 *  
 *  Main procedure - wo106  
 *  
*/
```

```

*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif

    void wol06( const int /* PlatePartNo */,
                const int /* ContourNo */,
                const int /* NumberOfCutouts */)
    {
    }
}

```

WO107 - AutoSeq Interface routine - Add Cutout/Notch for Contour

PURPOSE:

The procedure adds a cutout/notch for the current contour.

DECLARATION:

```
void wo107( const int PlatePartNo,  
            const int ContourNo,  
            const int CutoutNo,  
            const int SegPartStart,  
            const double StartPoint[2],  
            const int SegPartEnd,  
            const double EndPoint[2]);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
CutoutNo	The cutout number
SegPartStart	Segment part number for the cutout/notch start
StartPoint	Interval starting point
SegPartEnd	Segment part number for the cutout/notch end
EndPoint	Interval ending point

RESULT:

None.

```
-----*/  
/*  
*   External declarations
```

```

    */
extern "C"
{

    #if !defined TBLSEQUENCEPLUGIN_API
    #define TBSEQUENCEPLUGIN_API
    #endif

    /*

    ****
    ****
    *
    * Main procedure - wo107
    *

    ****
    ****
    */

    #ifdef WIN32
    TBSEQUENCEPLUGIN_API
    #endif
    void wo107( const int /* PlatePartNo */,
                const int /* ContourNo */,
                const int /* CutoutNo */,
                const int /* SegPartStart */,
                const double [2] /* StartPoint */,
                const int /* SegPartEnd */,
                const double [2] /* EndPoint */)
    {
    }
}

```


WO108

NAME:

WO108 - AutoSeq Interface routine - Create Automatic Sequence

PURPOSE:

The procedure creates the automatic sequence.

DECLARATION:

int wo108 (void);

INPUT:

RESULT:

The procedure returns 1 when the automatic sequence is finished, otherwise 0.

```
-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - wo108
 *
*****
 */
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        int wo108 ( void)
        {
            return 0;
        }
}
```

WO109

NAME:

WO109 - AutoSeq Interface routine - Get Number of Starts

PURPOSE:

The procedure gets the number of starts.

DECLARATION:

```
int wo109( void);
```

INPUT:

RESULT:

The number of nested starts is returned.

```
-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - wo109
 *
*****
 */
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        int wo109( void)
        {
            return 0;
        }
}
```

WO110

NAME:

WO110 - AutoSeq Interface routine - Get Start
Information

PURPOSE:

The procedure gets the information for a start.

DECLARATION:

```
void wo110( const int StartNo,  
            int& PlatePartNo,  
            int& StartType,  
            int& ContourNo,  
            int& SegNo,  
            double& Xpos,  
            double& Ypos);
```

INPUT:

Parameters:

StartNo	The identification number for the start
---------	--

RESULT:

Parameters:

PlatePartNo	The identification number of the plate part
-------------	--

StartType	The type of start: = 1 Burning = 2 Burning parallel = 3 Burning hole = 4 Burning hole parallel = 5 Blasting = 6 Blasting parallel = 7 Marking = 8 Marking parallel = 9 GSD = 10 GSD parallel
-----------	--

If a parallel start is mirrored
action, StartType < 0.

ContourNo	The contour number where the start
-----------	------------------------------------

	is placed
SegNo	The segment number where the start is placed
Xpos	The exact position for the start
Ypos	

REMARKS:

HOW TO HANDLE PARALLEL STARTS?????

```

-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
*
*   Main procedure - w0110
*
*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
void w0110( const int /* StartNo */,
            int& /* PlatePartNo */,
            int& /* StartType */,
            int& /* ContourNo */,
            int& /* SegNo */,
            double& /* Xpos */,
            double& /* Ypos */)
{
    {
    }
}

```

WO111

NAME:

WO111 - AutoSeq Interface routine - Cleanup

PURPOSE:

The procedure performs the necessary cleanup of local interface data structures.

DECLARATION:

void wo111 (void);

INPUT:

RESULT:

None.

```
-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - wo111
 *
*****
 */
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        void wo111 ( void)
        {
        }
}
```

WO112

NAME:

WO112 - AutoSeq Interface routine - Add Plate Part

PURPOSE:

The procedure adds a plate part.

DECLARATION:

```
void wo112( const int PlatePartNo,
            const int NumberOfContours,
            const char *PartName,
            const char *PositionNumber,
            const char *Quality,
            const double Thickness,
            const int QuantityNormal,
            const int QuantityMirror,
            const double PartLength,
            const double PartWidth,
            const double PartAngle,
            const double MinCircRectOrigin[2],
            const double Cog[2],
            const double Perimeter,
            const double EffectivePartArea,
            const double BoundaryPartArea,
            const char *Assid1,
            const char *Assid2,
            const char *Assid3,
            const char *Assid4,
            const char *Assembly,
            const char *ProductionDate,
            const double RotationAngle,
            const int MirrorFlag);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
NumberOfContours	The number of contours
PartName	The plate part name
PositionNumber	The plate part position number
Quality	The plate part quality
Thickness	The plate part thickness

QuantityNormal	The number of normal parts
QuantityMirror	The number of mirrored parts
PartLength	The length of the part along the longer side of the least circumscribed rectangle
PartWidth	The width of the part along the shorter side of the least circumscribed rectangle
MinCircRectOrigin	The origin of the least circumscribed rectangle
CoG	The centre of gravity of the plate part boundary
Perimeter	The perimeter of the plate part
EffectivePartArea	The plate part area with the area of the holes subtracted
BoundaryPartArea	The area of the plate part boundary
Assid1- Assid4	The assembly id's 1-4 from the General Purpose Strings (GPS)
Assembly	The plate part assembly
ProductionDate	The production date
RotationAngle	The rotation angle to be used in the nesting
MirrorFlag	The mirror flag: 0 = No mirroring allowed 1 = Mirroring allowed

RESULT:

None.

```

-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

```

```

/*
*****
*
*   Main procedure - w0112
*
*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif

void w0112( const int /* PlatePartNo */,
            const int /* NumberOfContours */,
            const char * /* PartName */,
            const char * /* PositionNumber */,
            const char * /* Quality */,
            const double /* Thickness */,
            const int /* QuantityNormal */,
            const int /* QuantityMirror */,
            const double /* PartLength */,
            const double /* PartWidth */,
            const double /* PartAngle */,
            const double [2] /* MinCircRectOrigin

*/,

            const double [2] /* Cog */,
            const double /* Perimeter */,
            const double /* EffectivePartArea */,
            const double /* BoundaryPartArea */,
            const char * /* Assid1 */,
            const char * /* Assid2 */,
            const char * /* Assid3 */,
            const char * /* Assid4 */,
            const char * /* Assembly */,
            const char * /* ProductionDate */,
            const double /* RotationAngle */,
            const int /* MirrorFlag */)

{
}

```


WO113

NAME:

WO113 - AutoSeq Interface routine - Add Segment Part
for Plate Part

PURPOSE:

The procedure adds one segment part for a plate part contour. The first segment part consists only of the starting point. Each successive segment part contains the amplitude vector and the ending point. 2 consecutive segment parts constitutes one segment.

DECLARATION:

```
void wo113( const int PartNo,  
            const int ContourNo,  
            const int SegPart,  
            const int SegType,  
            const double AmpU,  
            const double AmpV,  
            const double EndU,  
            const double EndV);
```

INPUT:

Parameters:

PartNo	The identification number for the plate part
ContourNo	The plate part contour number, starting from 1
SegPart	The segment part number, starting from 1
SegType	The segment part type: 0 = The starting point 1 = Line 2 = Arc
AmpU AmpV	The amplitude vector
EndU EndV	The ending point

RESULT:

None.

```
-----*/
/*
 *      External declarations
 */
extern "C"
{
#if !defined TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - w0113
 *
*****
 */
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
        void w0113( const int /* PartNo */,
                    const int /* ContourNo */,
                    const int /* SegPart */,
                    const int /* SegType */,
                    const double /* AmpU */,
                    const double /* AmpV */,
                    const double /* EndU */,
                    const double /* EndV */)
        {
        }
}
```

WO114

NAME:

WO114 - AutoSeq Interface routine - Define Number of
Bridges/Gaps

PURPOSE:

The procedure defines the number of bridges/gaps for
the current contour.

DECLARATION:

```
void wo114( const int PartNo,  
            const int ContourNo,  
            const int NumberOfBridges);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
NumberOfBridges	The number of bridges/gaps (only relevant for outer contours and holes).

RESULT:

None.

```
-----*/  
/*  
 *      External declarations  
 */  
extern "C"  
{  
#if !defined TBLSEQUENCEPLUGIN_API  
#define TBLSEQUENCEPLUGIN_API  
#endif  
  
/*  
*****  
 *  
 *  Main procedure - wo114  
 *  
*/
```

```
*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
    void w0114( const int /* PlatePartNo */,
                const int /* ContourNo */,
                const int /* NumberOfBridges */)
    {
    }
}
```

WO115

NAME:

WO115 - AutoSeq Interface routine - Add Bridge/Gap
for Contour

PURPOSE:

The procedure adds a bridge/gap for the current
contour.

DECLARATION:

```
void wo115( const int PlatePartNo,  
            const int ContourNo,  
            const int SegPartNo1,  
            const int SegPartNo2,  
            const double Pos1[2],  
            const double Pos2[2],  
            const int BridgeNo,  
            const int BridgeType,  
            const double BridgeWidth,  
            const int ConnPartNo,  
            const int ConnContourNo,  
            const int ConnSegPartNo1,  
            const int ConnSegPartNo2,  
            const double ConnPos1[2],  
            const double ConnPos2[2]);
```

INPUT:

Parameters:

PlatePartNo	The identification number for the plate part
ContourNo	The contour number
SegPartNo1	Segment part numbers for the bridge/gap
SegPartNo2	
Pos1	The end point of one of the bridge lines (start of gap)
Pos2	The end point of the other bridge line (end of gap)
BridgeNo	The bridge number
BridgeType	The bridge type:

= 0 Gap
 = 1 Double centered
 = 2 Single centered
 = 3 Double assymmetric
 = 4 Single assymmetric
 = 5 Cross centered
 = 6 Cross assymmetric
 = 7 As 3 but inside a cutout
 = 8 As 0, is used as start for
 a cut free geometry

For negative bridges, the type is negative

BridgeWidth	The bridge width
ConnPartNo	The identification number for the connecting plate part (bridge type 1-7 only)
ConnContourNo	The contour number for the connecting contour (bridge type 1-6 only)
ConnSegPartNo1 ConnSegPartNo2	The segment part numbers for the connecting contour (bridge type 1-6 only)
ConnPos1	The end point of one of the bridge lines for the connecting part
ConnPos2	The end point of the other bridge line for the connecting part

RESULT:

None.

```

-----*/
/*
 *      External declarations
 */
extern "C"
{
#ifdef TBLSEQUENCEPLUGIN_API
#define TBSEQUENCEPLUGIN_API
#endif

/*
*****
 *
 *   Main procedure - w0115
  
```

```

*
*****
*/
#ifdef WIN32
TBSEQUENCEPLUGIN_API
#endif
    void w0115( const int /* PlatePartNo */,
                const int /* ContourNo */,
                const int /* SegPartNo1 */,
                const int /* SegPartNo2 */,
                const double [2] /* Pos1 */,
                const double [2] /* Pos2 */,
                const int /* BridgeNo */,
                const int /* BridgeType */,
                const double /* BridgeWidth */,
                const int /* ConnPartNo */,
                const int /* ConnContourNo */,
                const int /* ConnSegPartNo1 */,
                const int /* ConnSegPartNo2 */,
                const double [2] /* ConnPos1 */,
                const double [2] /* ConnPos2 */)
    {
    }
}

```

Building the dll

Below follows an example of a project used for building the sample dll shown above in the Microsoft Visual Studio.

tbsequenceplugin.dsw

Microsoft Developer Studio Workspace File, Format Version 6.00
WARNING: DO NOT EDIT OR DELETE THIS WORKSPACE FILE!

#####

Project: "tbsequenceplugin"=".\\tbsequenceplugin.dsp" - Package
Owner=<4>

```
Package=<5>
{{{
    begin source code control
        tbsequenceplugin
    .
    end source code control
}}}
```

```
Package=<4>
{{{
}}}
```

#####

Global:

```
Package=<5>
{{{
    begin source code control
        tbsequenceplugin
    .
    end source code control
}}}
```

```
Package=<3>
{{{
}}}
```

tbsequenceplugin.dsp

Microsoft Developer Studio Project File - Name="tbsequenceplugin" - Package Owner=<4>
Microsoft Developer Studio Generated Build File, Format Version 6.00
** DO NOT EDIT **


```
# TARGETTYPE "Win32 (x86) Dynamic-Link Library" 0x0102
```

```
CFG=tbsequenceplugin - Win32 Debug
```

```
!MESSAGE This is not a valid makefile. To build this project using NMAKE,
```

```
!MESSAGE use the Export Makefile command and run
```

```
!MESSAGE
```

```
!MESSAGE NMAKE /f "tbsequenceplugin.mak".
```

```
!MESSAGE
```

```
!MESSAGE You can specify a configuration when running NMAKE
```

```
!MESSAGE by defining the macro CFG on the command line. For example:
```

```
!MESSAGE
```

```
!MESSAGE NMAKE /f "tbsequenceplugin.mak" CFG="tbsequenceplugin - Win32 Debug"
```

```
!MESSAGE
```

```
!MESSAGE Possible choices for configuration are:
```

```
!MESSAGE
```

```
!MESSAGE "tbsequenceplugin - Win32 Release" (based on "Win32 (x86) Dynamic-Link Library")
```

```
!MESSAGE "tbsequenceplugin - Win32 Debug" (based on "Win32 (x86) Dynamic-Link Library")
```

```
!MESSAGE
```

```
# Begin Project
```

```
# PROP AllowPerConfigDependencies 0
```

```
# PROP Scc_ProjName "tbsequenceplugin"
```

```
# PROP Scc_LocalPath "."
```

```
CPP=cl.exe
```

```
F90=df.exe
```

```
MTL=midl.exe
```

```
RSC=rc.exe
```

```
!IF "$(CFG)" == "tbsequenceplugin - Win32 Release"
```

```
# PROP BASE Use_MFC 0
```

```
# PROP BASE Use_Debug_Libraries 0
```

```
# PROP BASE Output_Dir "Release"
```

```
# PROP BASE Intermediate_Dir "Release"
```

```
# PROP BASE Target_Dir ""
```

```
# PROP Use_MFC 0
```

```
# PROP Use_Debug_Libraries 0
```

```
# PROP Output_Dir "Release"
```

```
# PROP Intermediate_Dir "Release"
```

```
# PROP Target_Dir ""
```

```
# ADD BASE F90 /compile_only /include:"Release/" /libs:dll /nologo /warn:nofileopt /dll
```

```
# ADD F90 /compile_only /include:"Release/" /libs:dll /nologo /warn:nofileopt /dll
```

```
# ADD BASE CPP /nologo /MT /W3 /GX /O2 /D "WIN32" /D "NDEBUG" /D "_WINDOWS" /D
```

```
"_MBCS" /D "_USRDLL" /D "TBLAYOUTPLUGIN_EXPORTS" /Yu"stdafx.h" /FD /c
```

```
# ADD CPP /nologo /MT /W3 /GX /O2 /D "WIN32" /D "NDEBUG" /D "_WINDOWS" /D "_MBCS"
```

```
/D "_USRDLL" /D "TBLAYOUTPLUGIN_EXPORTS" /FD /c
```

```
# SUBTRACT CPP /YX /Yc /Yu
```

```
# ADD BASE MTL /nologo /D "NDEBUG" /mktyplib203 /win32
```

```
# ADD MTL /nologo /D "NDEBUG" /mktyplib203 /win32
```

```
# ADD BASE RSC /I 0x409 /d "NDEBUG"
```

```
# ADD RSC /I 0x409 /d "NDEBUG"
```

```
BSC32=bscmake.exe
```

```
# ADD BASE BSC32 /nologo
```

```
# ADD BSC32 /nologo
```

```
LINK32=link.exe
```

```
# ADD BASE LINK32 kernel32.lib user32.lib gdi32.lib winspool.lib comdlg32.lib advapi32.lib
```

```
shell32.lib ole32.lib oleaut32.lib uuid.lib odbccp32.lib /nologo /dll /machine:I386
```

```
# ADD LINK32 kernel32.lib user32.lib gdi32.lib winspool.lib comdlg32.lib advapi32.lib shell32.lib
```

```
ole32.lib oleaut32.lib uuid.lib odbccp32.lib /nologo /dll /machine:I386
```

```

!ELSEIF "$(CFG)" == "tbsequenceplugin - Win32 Debug"

# PROP BASE Use_MFC 0
# PROP BASE Use_Debug_Libraries 1
# PROP BASE Output_Dir "Debug"
# PROP BASE Intermediate_Dir "Debug"
# PROP BASE Target_Dir ""
# PROP Use_MFC 0
# PROP Use_Debug_Libraries 1
# PROP Output_Dir "Debug"
# PROP Intermediate_Dir "Debug"
# PROP Ignore_Export_Lib 0
# PROP Target_Dir ""
# ADD BASE F90 /check:bounds /compile_only /debug:full /include:"Debug/" /libs:dll /nologo
/warn:argument_checking /warn:nofileopt /dll
# ADD F90 /check:bounds /compile_only /debug:full /include:"Debug/" /libs:dll /nologo
/warn:argument_checking /warn:nofileopt /dll
# ADD BASE CPP /nologo /MTd /W3 /Gm /GX /ZI /Od /D "WIN32" /D "_DEBUG" /D
"_WINDOWS" /D "_MBCS" /D "_USRDLL" /D "TBLAYOUTPLUGIN_EXPORTS" /Yu"stdafx.h"
/FD /GZ /c
# ADD CPP /nologo /MTd /W3 /Gm /GX /ZI /Od /D "WIN32" /D "_DEBUG" /D "_WINDOWS" /D
"_MBCS" /D "_USRDLL" /D "TBLAYOUTPLUGIN_EXPORTS" /FD /GZ /c
# SUBTRACT CPP /YX /Yc /Yu
# ADD BASE MTL /nologo /D "_DEBUG" /mktyplib203 /win32
# ADD MTL /nologo /D "_DEBUG" /mktyplib203 /win32
# ADD BASE RSC /I 0x409 /d "_DEBUG"
# ADD RSC /I 0x409 /d "_DEBUG"
BSC32=bscmake.exe
# ADD BASE BSC32 /nologo
# ADD BSC32 /nologo
LINK32=link.exe
# ADD BASE LINK32 kernel32.lib user32.lib gdi32.lib winspool.lib comdlg32.lib advapi32.lib
shell32.lib ole32.lib oleaut32.lib uuid.lib odbc32.lib odbccp32.lib /nologo /dll /debug /machine:I386
/pdbtype:sept
# ADD LINK32 kcsansts_d.lib kernel32.lib user32.lib gdi32.lib winspool.lib comdlg32.lib
advapi32.lib shell32.lib ole32.lib oleaut32.lib uuid.lib odbc32.lib odbccp32.lib /nologo /dll /debug
/machine:I386 /nodefaultlib:"libc.lib" /nodefaultlib:"libcmtd.lib" /nodefaultlib:"msvcrt.lib"
/nodefaultlib:"libcd.lib" /nodefaultlib:"msvcrt.lib" /pdbtype:sept /libpath:"\kcs\pvtlib\intel-nt\v4"
/libpath:"\kcs\intel-nt\v4\lib"

!ENDIF

# Begin Target

# Name "tbsequenceplugin - Win32 Release"
# Name "tbsequenceplugin - Win32 Debug"
# Begin Group "Source Files"

# PROP Default_Filter "cpp;c;cxx;rc;def;r;odl;idl;hpj;bat;f90;for;f;fpp"
# Begin Source File

SOURCE=.\tbsequenceplugin.cpp
# End Source File
# Begin Source File

SOURCE=.\tbsequenceplugin.def
# End Source File
# End Group
# Begin Group "Header Files"

```



```

const double StartNode,
const double BridgeWidth,
const double
    BridgeWidthHole,
const double
    StartPosition[2],
const double
    EndPosition[2],
const double
    EndMarkPosition[2],
const int MarkingFirst,
const int MarkingOpposite,
const double HoleStart,
const int HoleStartType);

TBSEQUENCEPLUGIN_API void wo103( const int PlatePartNo,
const int CntNo,
const int ContourNo,
const int
NumberOfSegParts,

const char *Assid1,
const char *Assid2,
const char *Assid3,
const char *Assid4,
const int ContourType,
const int MarkingType,
const int MarkingSide);

TBSEQUENCEPLUGIN_API void wo104( const int PlatePartNo,
const int ContourNo,
const int NumberOfBevels);

TBSEQUENCEPLUGIN_API void wo105( const int PlatePartNo,
const int ContourNo,
const int BevelIntervalNo,
const int SegPartStart,
const double
StartPoint[2],

const int SegPartEnd,
const double EndPoint[2],
const double BevelCode);

TBSEQUENCEPLUGIN_API void wo106( const int PlatePartNo,
const int ContourNo,
const int CutoutNo);

TBSEQUENCEPLUGIN_API void wo107( const int PlatePartNo,
const int ContourNo,
const int CutoutNo,
const int SegPartStart,
const double
StartPoint[2],

const int SegPartEnd,
const double EndPoint[2]);

TBSEQUENCEPLUGIN_API int wo108 ( void);

```

TBSEQUENCEPLUGIN_API	int wo109 (void);
TBSEQUENCEPLUGIN_API	void wo110(const int StartNo, int& PlatePartNo, int& StartType, int& ContourNo, int& SegNo, double& Xpos, double& Ypos);
TBSEQUENCEPLUGIN_API	void wo111(void);
TBSEQUENCEPLUGIN_API	void wo112(const int PlatePartNo, const int NumberOfContours, const char *PartName, const char *PositionNumber, const char *Quality, const double Thickness, const int QuantityNormal, const int QuantityMirror, const double PartLength, const double PartWidth, const double PartAngle, const double MinCircRectOrigin[2], const double Cog[2], const double Perimeter, const double EffectivePartArea, const double BoundaryPartArea, const char *Assid1, const char *Assid2, const char *Assid3, const char *Assid4, const char *Assembly, const char *ProductionDate, const double RotationAngle, const int MirrorFlag);
TBSEQUENCEPLUGIN_API	void wo113(const int PartNo, const int ContourNo, const int SegPart, const int SegType, const double AmpU, const double AmpV, const double EndU, const double EndV);
TBSEQUENCEPLUGIN_API	void wo114(const int PartNo, const int ContourNo,

```

                                const int
                                NumberOfBridges);

TBSEQUENCEPLUGIN_API void w0115( const int PlatePartNo,
                                const int ContourNo,
                                const int SegPartNo1,
                                const int SegPartNo2,
                                const double Pos1[2],
                                const double Pos2[2],
                                const int BridgeNo,
                                const int BridgeType,
                                const double BridgeWidth,
                                const int ConnPartNo,
                                const int ConnContourNo,
                                const int ConnSegPartNo1,
                                const int ConnSegPartNo2,
                                const double ConnPos1[2],
                                const double ConnPos2[2]);

#endif //
!defined(AFX_TBSEQUENCEPLUGIN_H__D6A40846_03F2_11D5_8393_08000
90A4C97__INCLUDED_)

```

tbsequenceplugin.cpp

```

// TBLayoutPlugin.cpp : Defines the entry point for the DLL
application.
//

#include "windows.h"
#include "tbsequenceplugin.h"

```

tbsequenceplugin.def

```

; TBLayoutPlugin.def : Declares the module parameters.

LIBRARY      "tbsequenceplugin.dll"

EXPORTS
    w0101
    w0102
    w0103
    w0104
    w0105
    w0106
    w0107
    w0108
    w0109
    w0110
    w0111

```

wo112
wo113
wo114
wo115